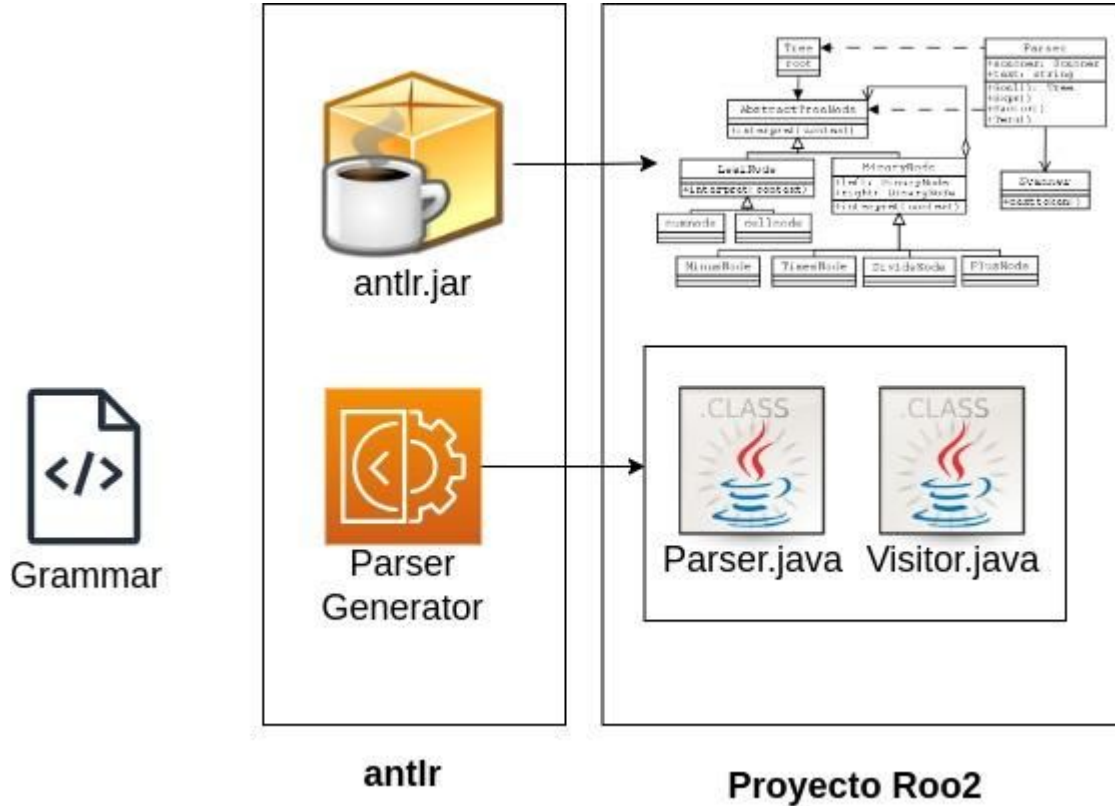


Cómo usar: antlr lab

<http://lab.antlr.org/>

...la última semana en Roo2



antlr lab

- Es una herramienta para
 - Prototipar lenguajes
 - Verificar definiciones de lenguajes
 - Testear la estructura del árbol generado por el parser
- Elementos a definir
 - Lexer
 - Conjunto de strings que forman los tokens del lenguaje
 - Parser
 - Reglas de derivación que conforman
 - Terminales
 - No Terminales
 - Orden de precedencia

1. Lexer tab
2. Parser tab
3. Panel editable
 - a. Lexer
 - b. Gramatica
4. Ejemplo
5. Árbol (CST)

1
2

Lexer
Parser
Sample

```

1 parser grammar ExprParser;
2 options { tokenVocab=ExprLexer; }
3
4 program
5     : stat EOF
6     | def EOF
7     ;
8
9 stat: ID '=' expr ';'
10     | ID DOT ID '=' expr ';'
11     | expr ';'
12     ;
13
14 def : ID '(' ID '(' ID ')' '*' ')' '{' stat* '}' ;
15
16 expr: ID
17     | INT
18     | func
19     | ID DOT ID
20     | ID DOT func
21     | 'not' expr
22     | expr 'and' expr
23     | expr 'or' expr
24     | expr (MUL | DIV) expr
25     | expr (SUM | SUB) expr
26     | expr POW expr
27     | expr '?' expr ':' expr
28     | expr '?' expr
29     ;
30
31 func : ID '(' expr '(' ID ')' '*' ')'
32     | ID '(' ' ' ')'
33     ;
34
  
```

Input sample.expr

```

1 f(x,y) {
2     a = 3+foo;
3     x and y;
4 }
  
```

Start rule program Run Show profiler

Tree Hierarchy

```

graph TD
    program2["program:2"] --> def1["def:1"]
    program2 --> eof["<EOF>"]
    def1 --> fxy["f ( x , y ) {"]
    def1 --> stat1["stat:1"]
    def1 --> stat3["stat:3 }"]
    stat1 --> aeq["a ="]
    stat1 --> expr10["expr:10"]
    stat1 --> semicolon1[";"]
    expr10 --> expr2["expr:2"]
    expr10 --> plus["+"]
    expr10 --> expr1_10["expr:1"]
    expr2 --> 3["3"]
    expr1_10 --> foo["foo"]
    stat3 --> expr7["expr:7"]
    stat3 --> and1["and"]
    stat3 --> expr1_3["expr:1"]
    expr7 --> expr1_7["expr:1"]
    expr7 --> and2["and"]
    expr7 --> expr1_7_2["expr:1"]
    expr1_7 --> x["x"]
    expr1_7_2 --> y["y"]
  
```

Pasos

1. Descargue la definición provista por la materia “pool.g4”
2. Parser
 - a. Click en el Parser tab (2)
 - b. Copie el contenido del archivo y pegue el texto en el panel editable (3)



Welcome to the ANTLR lab, where you can learn about [ANTLR](#) or experiment with and test grammars! Just hit the [Run](#) button to try out the sample grammar.

To start developing with ANTLR, see [getting started](#).

[Feedback/issues](#)
welcome. Brought to you by [Terence Parr](#), the maniac behind ANTLR.

Disclaimer: This website and related functionality are not meant to be used for private code, data, or other intellectual property. Assume everything you enter could become public! Grammars and input you enter are submitted to a unix box for execution and possibly persisted on disk or other mechanism. Please run antlr4-lab locally to avoid privacy concerns.

Lexer Parser

```
1 grammar pool;
2
3 behaviour
4 : def* EOF
5 ;
6
7 def : ID '(' (ID '(' ID)*)? ')' '{' stat* '}' ;
8
9 stat: ID '=' expr ';'
10 | ID DOT ID '=' expr ';'
11 | expr ';'
12 ;
13
14 expr: ID # AtomId
15 | INT # AtomInt
16 | BOOL # AtomBool
17 | 'this' # AtomThis
18 | func # AtomFunc
19 | 'new' ID '(' (expr '(' expr)*)? ')' # NewObj
20 | ID DOT ID # FieldAccess
21 | ID DOT func # MethodCall
22 | 'this' DOT ID # ThisFieldAccess
23 | 'this' DOT func # ThisMethodCall
24 | LPAREN expr RPAREN # Paren
25 | SUB expr # UnaryMinus
26 | NOT expr # UnaryNot
27 | <assoc=right> expr POW expr # Power
28 | expr (MUL | DIV) expr # MulDiv
29 | expr (SUM | SUB) expr # AddSub
30 | expr (LT | GT | LE | GE | EQEQ | NEQ) expr # Compare
31 | expr AND expr # LogicAnd
32 | expr OR expr # LogicOr
33 | expr ASK expr COLON expr # Ternary
34 ;
35
36 func : ID '(' (expr '(' expr)*)? ')' ;
37
38 AND : 'and' ;
39 BOOL : 'true' | 'false' ;
40 OR : 'or' ;
41 NOT : 'not' ;
42 EQ : '=' ;
43 EQEQ : '==' ;
44 NEQ : '!=' ;
45 LE : '<=' ;
46 GE : '>=' ;
47 LT : '<' ;
48 GT : '>' ;
49 COMMA : ',' ;
50 SEMI : ';' ;
51 LPAREN : '(' ;
52 RPAREN : ')' ;
53 LCURLY : '{' ;
54 RCURLY : '}' ;
55 SUM : '+' ;
56 SUB : '-' ;
57 MUL : '*' ;
58 DIV : '/' ;
59 POW : '^' ;
60 DOT : '.' ;
61 COLON : ':' ;
62 ASK : '?' ;
63 INT : [0-9]+ ;
64 ID : [a-zA-Z_][a-zA-Z_0-9]* ;
65 WS : [ \t\n\r\f]+ -> skip ;
```

Input

```
1 f(x,y) {
2     a = 3+foo;
3     x and y;
4 }
```

Start rule

program [Run](#)

Pasos

1. Descargue la definición provista por la materia “pool.g4”
2. Parser
 - a. Click en el Parser tab (2)
 - b. Copie el contenido del archivo y pegue el texto en el panel editable (3)
 - c. **Elimine el contenido del tab Lexer (1)**

Lexer Parser Sample ?

```
1 // DELETE THIS CONTENT IF YOU PUT COMBINED GRAMMAR IN Parser TAB
2 lexer grammar ExprLexer;
3
4 AND : 'and' ;
5 OR : 'or' ;
6 NOT : 'not' ;
7 EQ : '=' ;
8 COMMA : ',' ;
9 SEMI : ';' ;
10 LPAREN : '(' ;
11 RPAREN : ')' ;
12 LCURLY : '{' ;
13 RCURLY : '}' ;
14
15 INT : [0-9]+ ;
16 ID : [a-zA-Z_][a-zA-Z_0-9]* ;
17 WS : [ \t\n\r\f]+ -> skip ;
```

Lexer Parser Sample ?



1

Pasos

1. Descargue la definición provista por la materia “pool.g4”
2. Parser
 - a. Click en el Parser tab (2)
 - b. Copie el contenido del archivo y pegue el texto en el panel editable (3)
 - c. Elimine el contenido del tab Lexer (1)

Start Rule



ANTLR lab, where you can learn about ANTLR or experiment with and test grammars! Just hit the Run button to try out the sample grammar.

To start developing with ANTLR, see [getting started](#).

[Feedback/issues](#) welcome. Brought to you by [Terence Parr](#), the maniac behind ANTLR.

Disclaimer: This website and related functionality are not meant to be used for private code, data, or other intellectual property. Assume everything you enter could become public! Grammars and input you enter are submitted to a unix box for execution and possibly persisted on disk or other mechanism. Please run antlr4-tab locally to avoid privacy concerns.

Lexer Parser

```
1 grammar pool;
2
3 behaviour
4   : def* EOF
5   ;
6
7 def : ID '(' (ID '(' ID)*? ')' '{' stat* '}' ;
8
9 stat: ID '=' expr ';'
10    | ID DOT ID '=' expr ';'
11    | expr ';'
12    ;
13
14 expr: ID                                     # AtomId
15     | INT                                   # AtomInt
16     | BOOL                                 # AtomBool
17     | 'this'                               # AtomThis
18     | func                                  # AtomFunc
19     | 'new' ID '(' (expr '(' expr)*? ')'   # NewObj
20     | ID DOT ID                             # FieldAccess
21     | ID DOT func                           # MethodCall
22     | 'this' DOT ID                         # ThisFieldAccess
23     | 'this' DOT func                       # ThisMethodCall
24     | LPAREN expr RPAREN                   # Paren
25     | SUB expr                             # UnaryMinus
26     | NOT expr                             # UnaryNot
27     | <assoc=right> expr POW expr          # Power
28     | expr (MUL | DIV) expr               # MulDiv
29     | expr (SUM | SUB) expr               # AddSub
30     | expr (LT | GT | LE | GE | EQEQ | NEQ) expr # Compare
31     | expr AND expr                       # LogicAnd
32     | expr OR expr                        # LogicOr
33     | expr ASK expr COLON expr            # Ternary
34     ;
35
36 func : ID '(' (expr '(' expr)*? ')' ;
37
38 AND : 'and' ;
39 BOOL : 'true' | 'false' ;
40 OR : 'or' ;
41 NOT : 'not' ;
42 EQ : '=' ;
43 EQEQ : '==' ;
44 NEQ : '!=' ;
45 LE : '<=' ;
46 GE : '>=' ;
47 LT : '<' ;
48 GT : '>' ;
49 COMMA : ',' ;
50 SEMI : ';' ;
51 LPAREN : '(' ;
52 RPAREN : ')' ;
53 LCURLY : '{' ;
54 RCURLY : '}' ;
55 SUM : '+' ;
56 SUB : '-' ;
57 MUL : '*' ;
58 DIV : '/' ;
59 POW : '^' ;
60 DOT : '.' ;
61 COLON : ':' ;
62 ASK : '?' ;
63 INT : [0-9]+ ;
64 ID : [a-zA-Z][a-zA-Z_0-9]* ;
65 WS : [ \t\n\r\f]+ -> skip ;
```

Input sample.expr

```
1 f(x,y) {
2   a = 3+foo;
3   x and y;
4 }
```

Start rule

program Run

Importante: Reemplazar program por behaviour

Pasos

1. Descargue la definición provista por la materia “pool.g4”
2. Parser
 - a. Click en el Parser tab (2)
 - b. Copie el contenido del archivo y pegue el texto en el panel editable (3)
3. Run
 - a. Escriba un ejemplo
 - b. Click en el boton (verde) Run
 - c. Si su “ejemplo” está OK, un árbol se dibujará abajo del botón
 - d. Si su “ejemplo” no está Ok, un árbol con indicaciones de error aparecerá abajo del botón.

Programa de ejemplo I

Prueba

```
f(x,y) {  
  a = estudiar and practicar and oo2;  
}
```

Programa de ejemplo II

Condicional simple ($\text{exp1} ? \text{exp2}$)
Condicional completo ($\text{exp1} ? \text{exp2} : \text{exp3}$)
Asignación y Condicionales

```
f(x,y) {  
  x ? x+1 ;  
  y ? y+ 1 : y-1;  
  z = y ? y+ 1 : y-1;  
}
```

Programa de ejemplo III

Envío de mensajes sin argumento Envío de mensajes con argumento Envío de mensajes con argumentos
<pre>f(x,y) { x.foo(); x.foo(4); x.bar(x.foo, 4); }</pre>